

Stage 1 实验报告

2022011223 熊泽恩

实验内容

Step 1

Step 1 没有需要修改的代码。我认真阅读了代码框架，大致了解了文件的组织结构和各个文件的作用。同时，我也通过阅读 [PLY \(Python Lex-Yacc\) 快速入门](#) 的部分章节，知道了 `ply` 该如何使用。

Step 2

我在 `tacgen.py` 和 `riscvasmemitter.py` 中的一元操作列表中添加了**按位非**和**逻辑非**两种运算，并且在 `RvUnaryOp` 和 `Riscv` 两个枚举类中添加了这两种一元运算符。

Step 3

Step 3 与 Step 2 是类似的。我在 `tacgen.py` 和 `riscvasmemitter.py` 中的二元操作列表中添加了**减法**、**乘法**、**除法**、**取模**四种运算，并且在 `RvUnaryOp` 和 `Riscv` 两个枚举类中添加了这四种二元运算符。

Step 4

Step 4 与 Step 2、Step 3 是类似的，继续向 `RvUnaryOp` 和 `Riscv` 两个枚举类中补充运算符即可。但 Step 4 与前面两个 step 也有不同：比较和逻辑运算，在 TAC 中可以用**一条指令实现**，但在 RISC-V 中，只有 `<` 和 `>` 是可以用**一条指令实现**的，其余指令都**至少需要用两条指令才能实现**。所以，在 `riscvasmemitter.py` 中，需要添加 `if-else` 语句，而不是单纯地补充操作列表。

逻辑与

用 `snez` 检查 `lhs` 是否为 0，并用 `sub` 将结果取相反数：此时如果 `lhs` 是 0，则结果为 -1；否则，结果为 0。

由于 -1 的二进制表示**为全一**，故此时再用 `and` 将此结果和 `rhs` 进行按位与运算，可以得到二者的逻辑与。

最后再用 `snez` 来判断结果是否非 0。

相等

用 `xor` 进行异或运算，然后用 `seqz` 判断结果是否为 0。

不等

用 `xor` 进行异或运算，然后用 `snez` 判断结果是否非 0。

大于等于

用 `slt` 判断是否小于，然后用 `seqz` 对结果取逻辑非。

小于等于

用 `sgt` 判断是否大于，然后用 `seqz` 对结果取逻辑非。

Stage 0 思考题

Step 1

第 1 题

在我们的框架中，从 AST 向 TAC 的转换经过了 `namer.transform`、`typer.transform` 两个步骤，如果没有这两个步骤，以下代码能正常编译吗，为什么？

```
1 | int main(){
2 |     return 10;
3 | }
```

去掉这两个步骤，题目中的代码仍能编译。这是因为 `namer.transform` 对应符号表构建的过程，`typer.transform` 对应类型检查的过程。题目中的代码并不涉及符号表构建，而类型检查也是正确的，所以跳过这两个步骤也能够成功通过编译。

第 2 题

我们的框架现在对于 `main` 函数没有返回值的情况是在哪一步处理的？报的是什么错？

对于 `main` 函数没有返回值的情况，在 `parser.parse` 中进行语法分析的时候会对其进行处理，已有框架返回的是 `syntax error` 这一错误。

我编写了以下程序：

```
1 | int main(){
2 |     return;
3 | }
```

得到了以下输出：

```
1 | $ python main.py --input example.c --parse
2 | syntax error: line 2, column 11
3 |     return;
4 | syntax error: line 3, column 1
5 | }
6 | syntax error: EOF
```

第 3 题

为什么框架定义了 `frontend/ast/tree.py:Unary`、`utils/tac/tacop.py:TacUnaryOp`、`utils/riscv.py:RvUnaryOp` 三种不同的一元运算符类型？

因为 AST 上的一元运算符、TAC 中的一元运算符，以及 RISC-V 语句中的一元运算符的定义是不一样的。AST 上的一元运算符对应着的是源程序代码，TAC 中的一元运算符代表三地址码中的一元操作，RISC-V 中的一元运算符表示 RISC-V 中仅含一个操作数的指令。

这些对应着的是不同的“一元运算符”；事实上，它们表示的领域和范围都不同。

Stage 1 思考题

Step 2

第 1 题

我们在语义规范中规定整数运算越界是未定义行为，运算越界可以简单理解成理论上的运算结果没有办法保存在 32 位整数的空间中，必须截断高于 32 位的内容。请设计一个 MiniDecaf 表达式，只使用 `--!` 这三个单目运算符和从 0 到 2147483647 范围内的非负整数，使得运算过程中发生越界。

`--2147483647` 符合题意，它在运算过程中会发生越界。原因是，`~2147483647` 的十六进制表示为 `0x7FFFFFFF`，取反后为 `0x80000000`，即为 32 位整数的最小值 -2147483648。然后将其取相反数，就会发生越界。

Step 3

第 1 题

我们知道“除数为零的除法是未定义行为”，但是即使除法的右操作数不是 0，仍然可能存在未定义行为。请问这时除法的左操作数和右操作数分别是什么？请将这时除法的左操作数和右操作数填入下面的代码中，分别在你的电脑（请标明你的电脑的架构，比如 x86-64 或 ARM）中和 RISC-V-32 的 qemu 模拟器中编译运行下面的代码，并给出运行结果。（编译时请不要开启任何编译优化）

```
1 #include <stdio.h>
2
3 int main() {
4     int a = 左操作数;
5     int b = 右操作数;
6     printf("%d\n", a / b);
7     return 0;
8 }
```

这是左操作数为 `-2147483648`，右操作数为 `-1`。

- 我的电脑（x86-64 架构，Windows 操作系统）中编译并运行的结果：无输出。
- RISC-V-32 的 qemu 模拟器中编译并运行的结果：`-2147483648`。

Step 4

第 1 题

在 MiniDecaf 中，我们对于短路求值未做要求，但在包括 C 语言的大多数流行的语言中，短路求值都是被支持的。为何这一特性广受欢迎？你认为短路求值这一特性会给程序员带来怎样的好处？

因为短路求值能够可以减少不必要的计算，从而提高程序的运行效率。如果第二个操作数的计算所需时间很长，则短路求值能够跳过这步无用的计算。

而且，在第二个操作数是函数调用的情况下，可能会有副作用（比如说在函数里修改了全局变量）。短路求值可以跳过那些操作，避免这些副作用的发生。

同时，短路求值这一特性可以给程序员带来很多编程下的便利。比如可以直接写：

```
1  for (int i = 0; i < n; i++) {  
2      int index = arr[i];  
3      if (index >= 0 && brr[index] == '#')  
4          // ...  
5  }
```

这样对于 `index >= 0` 的短路求值，可以保证 `brr[index]` 这样的访问必定是合法的。